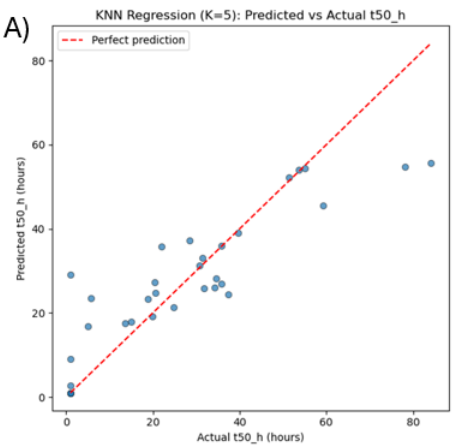
Creation of a K-nearest neighbour and linear regression ML model for predicting a drug's half life (t50h)

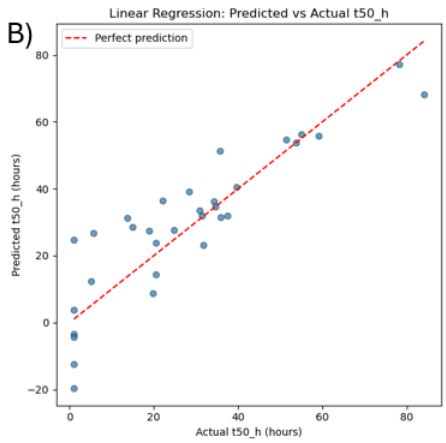
Robert Notingher

Dataset.py creates a synthetic dataset for 160 different drugs, each with 21 different properties. These 21 properties are derived from 7 latent factors for more realism and to generate correlations between features. The goal of this task was to select an outcome variable to predict with two different machine learning models. I selected a K-nearest neighbour (KNN) and a Lasso linear regression model to predict the half life of a drug (referred to in code as t50_h).

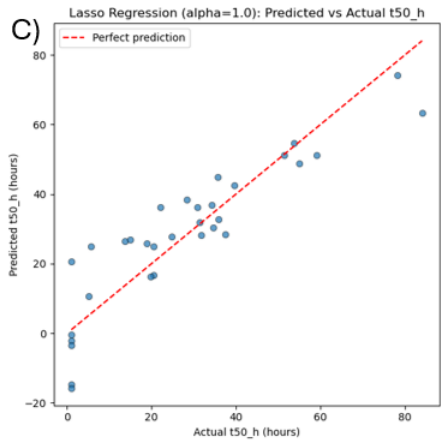
K-nearest neighbour algorithms are best known for their ability to classify a new data point by finding the nearest point(s) in the training set that're closest to the new entry before assigning that new entry the same class. "K" simply refers to the variability of the number of nearest neighbours (for example 3). However, K-nn can also be used for regression whereby the mean value of the nearest neighbours' t50_h to the new entry are calculated and used as a prediction for the new data point's t50_h. Meanwhile, linear regression models operate by creating a linear function to predict an outcome based off the data point's other features. For example, a linear regression model may be used to predict the 100m sprint times of a population by considering a range of other features such as height, lung capacity and BMI. In the case of our dataset, and trying to predict t50_h, the gap between the training set score and the testing set score was large. This suggests overfitting, meaning the prediction formula is too complicated partly due to the large number of features. Therefore, Lasso regularisation was implemented, simplifying the model by removing less important features and making it better suited to new data.




```
=== Model Performance ===
Metric                        Train     Test
-----------------------------------------------
R² Score                      0.858    0.759
Mean Absolute Error (hrs)      6.46     7.17
```




```
=== Model Performance ===
Metric                        Train     Test
-----------------------------------------------
R² Score                      0.869    0.772
Mean Absolute Error (hrs)      6.70     7.87
```




```
=== Model Performance ===
Metric                        Train     Test
-----------------------------------------------
R² Score                      0.852    0.811
Mean Absolute Error (hrs)      6.85     7.40
```

Figure 1: Model performance scores and graphs showing predicted vs actual t50_h values

A) KNN regression graph and prediction metrics. **B)** Simple linear regression graph and prediction metrics. **C)** Lasso regularised linear regression graph and prediction metrics. All made on Python.

A)

```
=== Feature Coefficients (most influential first) ===
                    Feature  Coefficient
            BurstRelease_%    -7.967937
                NP_Size_nm     4.894269
                     k_app    -4.301660
      MatrixHydrophobicity     4.126430
             PEG_Surface_%     3.026642
                       PDI     2.783012
                       HBA     2.748671
                       pKa     2.035723
                      tPSA    -1.690916
                   Zeta_mV    -1.536730
EncapsulationEfficiency_%     1.404955
                      logS    -1.156066
                     cLogP    -0.742125
          DrugLoading_wt%     0.608726
                      RotB     0.535630
                        MW     0.422828
            FinalRelease_%     0.421084
                       HBD    -0.347051
                 Release_pH    -0.192755
```

B)

```
=== Active Features (9 kept) ===
                    Feature  Coefficient
            BurstRelease_%    -7.986841
      MatrixHydrophobicity     5.343150
                     k_app    -3.815534
                NP_Size_nm     3.487067
             PEG_Surface_%     2.379224
                       PDI     1.726949
                   Zeta_mV    -0.644993
                       HBA     0.533338
            FinalRelease_%    -0.084549

=== Removed Features (10 zeroed out) ===
                    Feature  Coefficient
                         MW          0.0
                      cLogP          0.0
                        pKa          0.0
                       tPSA         -0.0
                        HBD          0.0
                       RotB          0.0
                       logS         -0.0
EncapsulationEfficiency_%          0.0
          DrugLoading_wt%          0.0
                 Release_pH         -0.0
```

Figure 2: Feature coefficient scores of different linear regression models

A) Simple linear regression feature coefficients. **B)** Lasso regularised linear regression feature coefficients. We can see that, of the 19 features, only 9 are considered worth of keeping.

All three of the ML models performed acceptably although the KNN model clearly was the least effective of the three with an R^2 score of 0.759 compared to 0.772 and 0.811. Furthermore, it appears to have the most significant overfitting as evidenced by a gap of 0.099 between the R^2 scores of the training and test sets. The simple linear regression ML model also has a somewhat large gap of 0.097, indicating another case of possible overfitting. This overfitting was what prompted me to try the Lasso model which not only had the highest R^2 score (0.811) but also had the smallest gap between training and test scores (0.041). Both linear regression models determine burst release percentage (BurstRelease_%) as being the most important feature for predicting t50_h. Runners up are nanoparticle size and matrix hydrophobicity for the simple and Lasso linear regression respectively.

Anthropic's Claude AI (Sonnet 4.6) was used for writing the majority of the code for this ML model.

A breakdown of the code will be provided on the next page as some supplementary information.

Until line 237, the code is identical to the Dataset.py file.

```
237  # ------------------------------------------------------
238  # STEP 1: Generate the dataset
239  # (paste your original make_dataset() code here, or import it)
240  # ------------------------------------------------------
241  # Assuming make_dataset() is defined above or imported, e.g.:
242  # from your_module import make_dataset
243
244  df = make_dataset(n=160, seed=42)
```

Here, the first new code begins, loading in the previously generated set.

```
246  # ------------------------------------------------------
247  # STEP 2: Define inputs (X) and target (y)
248  # ------------------------------------------------------
249  # We drop SampleID (just a label) and t50_h (what we're predicting)
250  X = df.drop(columns=["SampleID", "t50_h"])
251  y = df["t50_h"]
```

As a second step, Y is established as the target (output). All the features are considered as inputs (X) aside from SampleID (because it is just a name label) and t50_H as that is the output we are looking to predict.

```
254  # STEP 3: Split into training and test sets
255  # ------------------------------------------------------
256  X_train, X_test, y_train, y_test = train_test_split(
257      X, y, test_size=0.2, random_state=42
```

Next, the dataset is split into training and testing sets in an 8:2 ratio

```
261  # STEP 4: Scale the features
262  # ------------------------------------------------
263  scaler = StandardScaler()
264  X_train_scaled = scaler.fit_transform(X_train)
265  X_test_scaled  = scaler.transform(X_test)
```

As the features are on different scales, the function “StandardScaler()” has been used to standardise everything.

```
269  # ------------------------------------------------------
270  # STEP 5: Find the best alpha using cross-validation
271  # ------------------------------------------------------
272  alphas = [0.001, 0.005, 0.01, 0.05, 0.1, 0.5, 1.0, 5.0]
273  cv_scores = []
274
275  print("=== Cross-Validation Alpha Search ===")
276  print(f"{'Alpha':<10} {'Mean R²':>10} {'Std R²':>10}")
277  print("-" * 32)
278
279  for alpha in alphas:
280      lasso = Lasso(alpha=alpha, max_iter=10000)
281      scores = cross_val_score(lasso, X_train_scaled, y_train, cv=5, scoring="r2")
282      cv_scores.append(scores.mean())
283      print(f"{alpha:<10} {scores.mean():>10.3f} {scores.std():>10.3f}")
284
285  best_alpha = alphas[np.argmax(cv_scores)]
286  print(f"\nBest alpha: {best_alpha}")
```

Before training can begin, the Lasso model must select an alpha value which determines the intensity by which large coefficients are penalised. 6 different alpha values between 0.001 and 5 are tested with the best one being selected.

```python
# STEP 6: Train the final model with the best alpha
# ------------------------------------------------
model = Lasso(alpha=best_alpha, max_iter=10000)
model.fit(X_train_scaled, y_train)
```

In this case, an alpha value of 1.0 was selected to be the best

```python
# STEP 7: Evaluate the model on both train and test sets
# ------------------------------------------------------
y_pred_test  = model.predict(X_test_scaled)
y_pred_train = model.predict(X_train_scaled)

mae_test  = mean_absolute_error(y_test,  y_pred_test)
mae_train = mean_absolute_error(y_train, y_pred_train)
r2_test   = r2_score(y_test,  y_pred_test)
r2_train  = r2_score(y_train, y_pred_train)

print("\n=== Model Performance ===")
print(f"{'Metric':<26} {'Train':>8} {'Test':>8}")
print("-" * 44)
print(f"{'R² Score':<26} {r2_train:>8.3f} {r2_test:>8.3f}")
print(f"{'Mean Absolute Error (hrs)':<26} {mae_train:>8.2f} {mae_test:>8.2f}")
```

This section of code causes the R^2 and MAE values of the training and test sets to be printed side by side, allowing for easy comparison.

```python
# STEP 8: Inspect feature importance
# ----------------------------------------------------
coef_df = pd.DataFrame({
    "Feature":     X.columns,
    "Coefficient": model.coef_
}).sort_values("Coefficient", key=abs, ascending=False)

active   = coef_df[coef_df["Coefficient"] != 0]
removed  = coef_df[coef_df["Coefficient"] == 0]

print(f"\n=== Active Features ({len(active)} kept) ===")
print(active.to_string(index=False))

print(f"\n=== Removed Features ({len(removed)} zeroed out) ===")
print(removed.to_string(index=False))
```

The importance of the different features are printed as well as which ones are zeroed out entirely.

```python
# STEP 9: Plot predicted vs actual values
# ----------------------------------------------------
plt.figure(figsize=(6, 6))
plt.scatter(y_test, y_pred_test, alpha=0.7, edgecolors='k', linewidths=0.5)
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()],
         'r--', linewidth=1.5, label="Perfect prediction")
plt.xlabel("Actual t50_h (hours)")
plt.ylabel("Predicted t50_h (hours)")
plt.title(f"Lasso Regression (alpha={best_alpha}): Predicted vs Actual t50_h")
plt.legend()
plt.tight_layout()
plt.savefig("lasso_predicted_vs_actual.png", dpi=150)
plt.show()
print("\nPlot saved to lasso_predicted_vs_actual.png")
```